

PARTE 2 · CÓMO PENSAR UNA BASE DE DATOS

De cero absoluto a nivel experto



CLIENTES

id	nombre	email
1	Ana	ana@...
2	Luis	luis@...

```
SELECT nombre
FROM clientes
WHERE id = 1;

-- Tu 1ª consulta
```

BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Contenido de esta parte

Antes de empezar

2.1 De la idea a los datos

2.2 Cómo detectar entidades, atributos, relaciones y reglas

2.3 Datos obligatorios y datos opcionales

2.4 Caso guiado: «Quiero crear un CRM»

2.5 Otros casos de entrenamiento

2.6 Plantilla reutilizable de análisis de requisitos de datos

Resumen de la Parte 2

Glosario de la Parte 2

Mini examen de la Parte 2

Parte 2. Cómo pensar una base de datos

Antes de empezar

En la Parte 1 aprendiste qué es una tabla y de qué se compone. Ahora viene lo más importante: **saber qué tablas hacen falta** cuando alguien te cuenta una idea. Nadie empieza una base de datos escribiendo código. Primero se piensa, se describe y se dibuja (**sin código**, con papel y lápiz); el código viene después y es la parte fácil.

Por eso cada punto de esta parte tiene **dos mitades**:

1. **Primero, sin código**: el razonamiento y las tablas hechas a mano, con datos de ejemplo.
2. **Después, con código**: el mismo diseño escrito en SQL y ejecutado en PostgreSQL, con su captura real.

Al terminar serás capaz de:

- convertir una idea escrita («quiero una tienda», «quiero un CRM») en una lista de cosas que hay que guardar;
- distinguir entidades, atributos, relaciones y reglas dentro de una descripción;
- decidir qué datos son obligatorios, cuáles opcionales y cuáles únicos;
- construir las tablas de un CRM completo a mano y después en SQL;
- reconocer patrones habituales (agenda, tienda, reservas);
- rellenar una plantilla de requisitos de datos reutilizable.

Palabras nuevas de esta parte

Palabra	Significado breve
Entidad	Una «cosa» de la que quieres guardar datos: cliente, pedido, producto. Cada entidad acabará siendo una tabla
Atributo	Una característica de una entidad: el nombre de un cliente, el precio de un producto. Cada atributo acabará siendo una columna
Relación	La conexión entre dos entidades: un cliente hace pedidos
Regla	Una condición que los datos deben cumplir: el precio no puede ser negativo

Cómo se organiza cada punto

Cada punto sigue los mismos pasos: **¿qué es?**, **¿para qué sirve?**, **¿por qué lo necesito?**, **¿cómo funciona?**, **primero sin código**, **paso a paso con código**, **código y resultado** (con captura real), ejemplos, **profundizando**, ejercicio, solución, error habitual, buena práctica y comprobación.

Las capturas con fondo oscuro son la salida real de `psql` en PostgreSQL 16.14. Si aún no tienes `psql`, haz antes la Parte 0.

2.1 De la idea a los datos

¿Qué es?

Es el método que usan los profesionales para empezar cualquier base de datos: **antes de pensar en tablas, hay que entender qué se quiere guardar y para qué**. Se parte de una frase corriente («quiero una tienda online») y se acaba con una lista ordenada de cosas, características, conexiones y reglas.

¿Para qué sirve?

Para no empezar a ciegas. Quien crea las tablas «a ojo» suele descubrir a mitad del proyecto que le falta información, que algo está duplicado o que no puede responder una pregunta básica (por ejemplo, «¿qué libros tiene este amigo ahora?»). Cambiar la estructura con datos ya guardados es mucho más costoso que pensarla bien al principio.

¿Por qué lo necesito?

Porque la estructura de la base de datos condiciona todo lo demás: las consultas que podrás hacer, lo rápido que irá la aplicación y lo fácil que será ampliarla. Un diseño pensado con calma ahorra semanas de arreglos; uno improvisado se paga con intereses.

¿Cómo funciona?

El método tiene seis pasos. Los repetirás en cada proyecto:

1. **Describe la idea con frases completas.** Cuéntala como si se la explicaras a un amigo.
2. **Subraya los sustantivos importantes.** Son candidatos a **entidades**.
3. **Anota qué se sabe de cada uno.** Son los **atributos**.
4. **Busca los verbos que unen entidades.** Son las **relaciones**.
5. **Escribe las reglas.** Lo que siempre debe cumplirse.
6. **Revisa con preguntas reales.** Comprueba que con esos datos podrías responder lo que el negocio necesita.



Figura 2.1-a. Los seis pasos del método para pasar de una idea a los datos.

Qué ves en la imagen. Seis estaciones de izquierda a derecha. La flecha de vuelta indica que, si la revisión falla, se regresa a los pasos anteriores: diseñar es iterar.

Primero, sin código: la idea del préstamo de libros, a mano

Idea: «Quiero guardar los libros que presto a mis amigos».

1. **Frase completa:** «Presto libros a amigos. Quiero saber quién tiene cada libro y desde cuándo».
2. **Sustantivos importantes (entidades):** amigo, libro, préstamo.
3. **Atributos:** amigo (nombre, teléfono); libro (título, autor); préstamo (fecha de salida, fecha de devolución).
4. **Relaciones:** un amigo recibe préstamos; un préstamo es de un libro.
5. **Reglas:** un libro no puede estar prestado a dos amigos a la vez; la fecha de devolución es opcional (se rellena al devolver).
6. **Revisión con una pregunta:** «¿qué libros tiene Marta ahora?». Se responde mirando los préstamos de Marta que **no** tienen fecha de devolución. ¿Hay datos suficientes? Sí.

Paso a paso: ahora con código

1. **Conéctate a la base de datos** `ensayo` (`\c ensayo`).
2. **Convierte cada entidad en una tabla** con `CREATE TABLE` , y cada atributo en una columna con su tipo.
3. **Convierte las relaciones en claves extranjeras** (`REFERENCES`): el préstamo apunta al amigo y al libro.
4. **Inserta datos de ejemplo** (los mismos que usarías en papel).
5. **Haz la pregunta del paso 6** con una consulta y comprueba que da la respuesta esperada.

Código y resultado

```
CREATE TABLE amigos (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  telefono text
);

CREATE TABLE libros (
  id      integer PRIMARY KEY,
  titulo  text NOT NULL,
  autor   text
);

CREATE TABLE prestamos (
  id              integer PRIMARY KEY,
  amigo_id       integer NOT NULL REFERENCES amigos (id),
  libro_id       integer NOT NULL REFERENCES libros (id),
  fecha_salida   date NOT NULL,
  fecha_devolucion date
);
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE amigos (
ensayo-#   id          integer PRIMARY KEY,
ensayo-#   nombre     text NOT NULL,
ensayo-#   telefono   text
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE libros (
ensayo-#   id          integer PRIMARY KEY,
ensayo-#   titulo     text NOT NULL,
ensayo-#   autor      text
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE prestamos (
ensayo-#   id          integer PRIMARY KEY,
ensayo-#   amigo_id    integer NOT NULL REFERENCES amigos (id),
ensayo-#   libro_id    integer NOT NULL REFERENCES libros (id),
ensayo-#   fecha_salida date NOT NULL,
ensayo-#   fecha_devolucion date
ensayo-# );
CREATE TABLE

```

Figura 2.1-b. Captura real: las tres entidades del préstamo de libros convertidas en tablas.

Qué ves en la imagen. Cada entidad es un `CREATE TABLE`. Los atributos son las columnas (`nombre text NOT NULL` es un texto obligatorio). `prestamos` guarda dos enlaces, `amigo_id` y `libro_id`, declarados con `REFERENCES`: así cada préstamo apunta a un amigo y a un libro que existen. `fecha_devolucion` no lleva `NOT NULL`: es opcional, como decidimos. Cada tabla responde `CREATE TABLE`.

```

INSERT INTO amigos VALUES (1, 'Marta', '600333444'), (2, 'Pablo', NULL);
INSERT INTO libros  VALUES (1, 'Cien años de soledad', 'García Márquez'), (2,
'Rayuela', 'Cortázar');
INSERT INTO prestamos VALUES
  (1, 1, 1, '2026-03-01', NULL),
  (2, 2, 2, '2026-02-10', '2026-02-25');

SELECT a.nombre AS amigo, l.titulo
FROM prestamos p
JOIN amigos a ON a.id = p.amigo_id
JOIN libros l ON l.id = p.libro_id
WHERE p.fecha_devolucion IS NULL;

```

```

psql — base de datos «ensayo»

ensayo=# INSERT INTO amigos VALUES (1, 'Marta', '600333444'), (2, 'Pablo', NULL);
ensayo=# INSERT INTO libros VALUES (1, 'Cien años de soledad', 'García Márquez'), (2, 'Rayuela', 'Cortázar');
ensayo=# INSERT INTO prestamos VALUES
ensayo-# (1, 1, 1, '2026-03-01', NULL),
ensayo-# (2, 2, 2, '2026-02-10', '2026-02-25');
INSERT 0 2
INSERT 0 2
INSERT 0 2

ensayo=# SELECT a.nombre AS amigo, l.titulo
ensayo-# FROM prestamos p
ensayo-# JOIN amigos a ON a.id = p.amigo_id
ensayo-# JOIN libros l ON l.id = p.libro_id
ensayo-# WHERE p.fecha_devolucion IS NULL;
 amigo |          titulo
-----+-----
 Marta | Cien años de soledad
(1 row)

```

Figura 2.1-c. Captura real: datos de ejemplo y la pregunta «¿qué libros están prestados ahora?».

Qué ves en la imagen. Los tres `INSERT` guardan dos amigos, dos libros y dos préstamos (el de Pablo ya fue devuelto; el de Marta, no). La consulta une las tres tablas con `JOIN` (se estudia en la Parte 5) y filtra con `WHERE fecha_devolucion IS NULL`: solo aparece Marta con *Cien años de soledad*, que es la respuesta correcta a la pregunta de negocio.

Ejemplo sencillo

«Quiero apuntar los gastos de mi casa»: entidades *gasto* y *categoría*; relación: cada gasto es de una categoría; pregunta: «¿cuánto gastamos en comida en marzo?».

Ejemplo real

Un dueño de gimnasio dice: «Quiero controlar quién está apuntado a cada clase y cuándo paga cada socio». Salen entidades como socio, clase, inscripción y pago, y preguntas que nadie se había planteado: ¿un socio puede apuntarse a varias clases? ¿Una clase tiene aforo máximo? ¿Se guarda el historial de pagos? Cada pregunta sin respuesta es una decisión de diseño pendiente.

Profundizando

El diseño es iterativo. Rara vez el primer diseño es el definitivo. Se revisa con ejemplos reales: si una fila de ejemplo no cabe, se corrige el diseño.

Pregunta siempre por los datos que ya existen. Si el cliente ya tiene una hoja de cálculo, es la mejor fuente de entidades y atributos; sus columnas suelen delatar tablas escondidas.

Cuando algo «cabe» en dos sitios. Si dudas entre guardar un dato como atributo o como entidad, pregúntate si hay muchos y si tiene datos propios (punto 2.2).

Ejercicio

1. Aplica los seis pasos a esta idea: «Quiero una aplicación para apuntar los gastos de mi casa». Escribe entidades, atributos, relaciones, una regla y una pregunta de revisión.
2. Escribe las tablas en SQL (solo los `CREATE TABLE`).
3. ¿Por qué conviene empezar con frases corrientes y no con nombres de tablas?

Solución

1. Entidades: *gasto* y *categoría*. Atributos de gasto: fecha, importe, descripción. De categoría: nombre. Relación: cada gasto pertenece a una categoría. Regla: el importe no es negativo. Pregunta: «¿cuánto gastamos en comida en marzo?».
2. Por ejemplo:

```
CREATE TABLE categorias (
  id      integer PRIMARY KEY,
  nombre text NOT NULL UNIQUE
);
CREATE TABLE gastos (
  id            integer PRIMARY KEY,
  categoria_id integer NOT NULL REFERENCES categorias (id),
  fecha         date NOT NULL,
  importe       numeric(10, 2) NOT NULL CHECK (importe >= 0),
  descripcion   text
);
```

1. Porque las frases corrientes obligan a pensar en el negocio y no en la técnica. Si empiezas por las tablas, tiendes a copiar estructuras que no encajan con tu caso.

Error habitual

Saltarse el análisis y crear tablas directamente («ya iré arreglándolo»). Con datos ya guardados, cada cambio de estructura obliga a migrar información y a modificar la aplicación.

```
-- Mal: todo en una sola tabla, sin pensar en las entidades
CREATE TABLE todo_junto (amigo text, telefono text, libro text, autor text,
salida date, devolucion date);
```

Buena práctica

Escribe la descripción del proyecto en un documento antes de abrir ninguna herramienta, y enséñasela a la persona que usará el sistema: si no la reconoce, falta algo.

Comprobación

Para un proyecto tuyo, escribe en tres frases qué quieres guardar y tres preguntas que el sistema debe responder. Si no sabes las preguntas, todavía no estás listo para diseñar. Después, comprueba con datos de ejemplo que tu diseño las responde.

2.2 Cómo detectar entidades, atributos, relaciones y reglas

¿Qué es?

Es la técnica de **leer una descripción y clasificar sus palabras** en cuatro grupos:

- **Entidades:** las cosas de las que se guardan datos. Suelen ser sustantivos: cliente, pedido, producto.
- **Atributos:** las características de cada entidad: el *nombre* del cliente, el *precio* del producto.
- **Relaciones:** cómo se conectan las entidades. Suelen ser verbos: un cliente *hace* pedidos, un pedido *incluye* productos.
- **Reglas:** condiciones que los datos deben cumplir. Suelen aparecer con palabras como «siempre», «nunca», «como máximo», «no puede», «único».

¿Para qué sirve?

Convierte un texto ambiguo en una lista concreta. Es la manera más fiable de no olvidar nada y de que dos personas que lean la misma descripción lleguen a la misma estructura.

¿Por qué lo necesito?

Muchos errores de diseño nacen de clasificar mal: guardar la *ciudad* como entidad cuando es solo un dato, o guardar el *pedido* como un atributo del cliente cuando es una entidad con vida propia.

¿Cómo funciona?

Para decidir si un sustantivo es una entidad o un atributo, hazte tres preguntas:

1. **¿Quiero guardar varios datos sobre ello?** Si sí, es probablemente una entidad.
2. **¿Puede haber muchos?** Un cliente puede tener muchos pedidos (entidad); un cliente tiene un solo nombre (atributo).
3. **¿Existe por sí mismo?** Un pedido existe aunque cambie el cliente; el nombre no existe sin el cliente.

Para detectar relaciones, busca verbos entre dos entidades. Para detectar reglas, busca palabras de restricción.

Primero, sin código: clasificar una descripción a mano

Descripción: «Una tienda vende productos. Cada cliente puede hacer varios pedidos. Un pedido incluye uno o varios productos. De cada cliente guardamos nombre, email y teléfono. El email es obligatorio y no puede repetirse; el teléfono es opcional. Un producto tiene nombre y precio, y el precio no puede ser negativo».

1. **Lee la descripción** y subraya con cuatro colores: entidades, atributos, relaciones y reglas.
2. **Pasa las palabras a cuatro listas.**

3. Comprueba cada entidad con las tres preguntas.

Una tienda vende productos. Cada cliente puede hacer varios pedidos. Un pedido incluye uno o varios productos. De cada cliente guardamos nombre, email y teléfono. El email es obligatorio y no puede repetirse; el teléfono es opcional. Un producto tiene nombre y precio, y el precio no puede ser negativo.

☐ Entidades
 ☐ Atributos
 ☐ Relaciones
 ☐ Reglas

Figura 2.2-a. Una descripción con sus palabras clasificadas por colores.

Qué ves en la imagen. El texto de la tienda con las palabras importantes coloreadas: azul para las entidades, naranja para los atributos, morado para las relaciones y verde para las reglas. Subrayar así es el primer paso de cualquier análisis.

Entidades → tablas	Atributos → columnas	Relaciones → enlaces	Reglas → restricciones
cliente pedido producto	cliente: nombre, email, teléfono producto: nombre, precio	cliente hace pedidos pedido incluye productos	email obligatorio y único teléfono opcional precio no negativo

Figura 2.2-b. El resultado de la clasificación, en cuatro cajas.

Qué ves en la imagen. Las mismas palabras ya ordenadas en cuatro cajas: lo que acabará siendo tablas (entidades), columnas (atributos), enlaces entre tablas (relaciones) y restricciones (reglas). Esta lista es el material de partida del diseño de la Parte 3.

Paso a paso: ahora con código

1. Cada entidad es una tabla: `clientes`, `productos`, `pedidos`.
2. Cada atributo es una columna con su tipo.
3. Cada relación es un enlace (`REFERENCES`). Las relaciones «muchos a muchos» necesitan una tabla intermedia (punto 3.3): aquí `lineas_pedido`.
4. Cada regla es una restricción: `NOT NULL`, `UNIQUE`, `CHECK`.
5. Comprueba con `\dt` que están todas las tablas.

Código y resultado

```
-- Entidades -> tablas; atributos -> columnas
CREATE TABLE clientes (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  email   text NOT NULL UNIQUE,      -- regla: obligatorio y único
  telefono text                      -- regla: opcional
);

CREATE TABLE productos (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  precio  numeric(10, 2) NOT NULL CHECK (precio >= 0)  -- regla: no negativo
);

-- Relaciones: el cliente HACE pedidos; el pedido INCLUYE productos
CREATE TABLE pedidos (
  id          integer PRIMARY KEY,
  cliente_id  integer NOT NULL REFERENCES clientes (id)
);

CREATE TABLE lineas_pedido (
  pedido_id  integer REFERENCES pedidos (id),
  producto_id integer REFERENCES productos (id),
  cantidad   integer NOT NULL CHECK (cantidad > 0),
  PRIMARY KEY (pedido_id, producto_id)
);
```

```

psql — base de datos «tienda»

tienda=# -- Entidades -> tablas; atributos -> columnas
tienda=# CREATE TABLE clientes (
tienda=#   id         integer PRIMARY KEY,
tienda=#   nombre    text NOT NULL,
tienda=#   email     text NOT NULL UNIQUE,      -- regla: obligatorio y único
tienda=#   telefono  text                      -- regla: opcional
tienda=# );
CREATE TABLE

tienda=# CREATE TABLE productos (
tienda=#   id         integer PRIMARY KEY,
tienda=#   nombre    text NOT NULL,
tienda=#   precio    numeric(10, 2) NOT NULL CHECK (precio >= 0) -- regla: no negativo
tienda=# );
CREATE TABLE

tienda=# -- Relaciones: el cliente HACE pedidos; el pedido INCLUYE productos
tienda=# CREATE TABLE pedidos (
tienda=#   id         integer PRIMARY KEY,
tienda=#   cliente_id integer NOT NULL REFERENCES clientes (id)
tienda=# );
CREATE TABLE

tienda=# CREATE TABLE lineas_pedido (
tienda=#   pedido_id integer REFERENCES pedidos (id),
tienda=#   producto_id integer REFERENCES productos (id),
tienda=#   cantidad  integer NOT NULL CHECK (cantidad > 0),
tienda=#   PRIMARY KEY (pedido_id, producto_id)
tienda=# );
CREATE TABLE

```

Figura 2.2-c. Captura real: las cuatro listas de la clasificación escritas como SQL. Los comentarios (--) indican qué regla de la descripción implementa cada línea.

Qué ves en la imagen. Cada elemento de la descripción tiene su equivalente: las entidades se han convertido en las cuatro tablas; los atributos, en columnas; «el cliente hace pedidos» en `cliente_id ... REFERENCES clientes (id)`; «el email es obligatorio y no puede repetirse» en `email text NOT NULL UNIQUE`; «el teléfono es opcional» en `telefono text` (sin `NOT NULL`); y «el precio no puede ser negativo» en `CHECK (precio >= 0)`. La relación «un pedido incluye productos» se resuelve con la tabla `lineas_pedido`.

```
\dt
```

```

psql — base de datos «tienda»

tienda=# \dt
          List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | clientes       | table | postgres
public | lineas_pedido  | table | postgres
public | pedidos       | table | postgres
public | productos     | table | postgres
(4 rows)

```

Figura 2.2-d. Captura real: las cuatro tablas creadas.

Qué ves en la imagen. `\dt` lista las tablas de `tienda`: `clientes`, `lineas_pedido`, `pedidos` y `productos`.

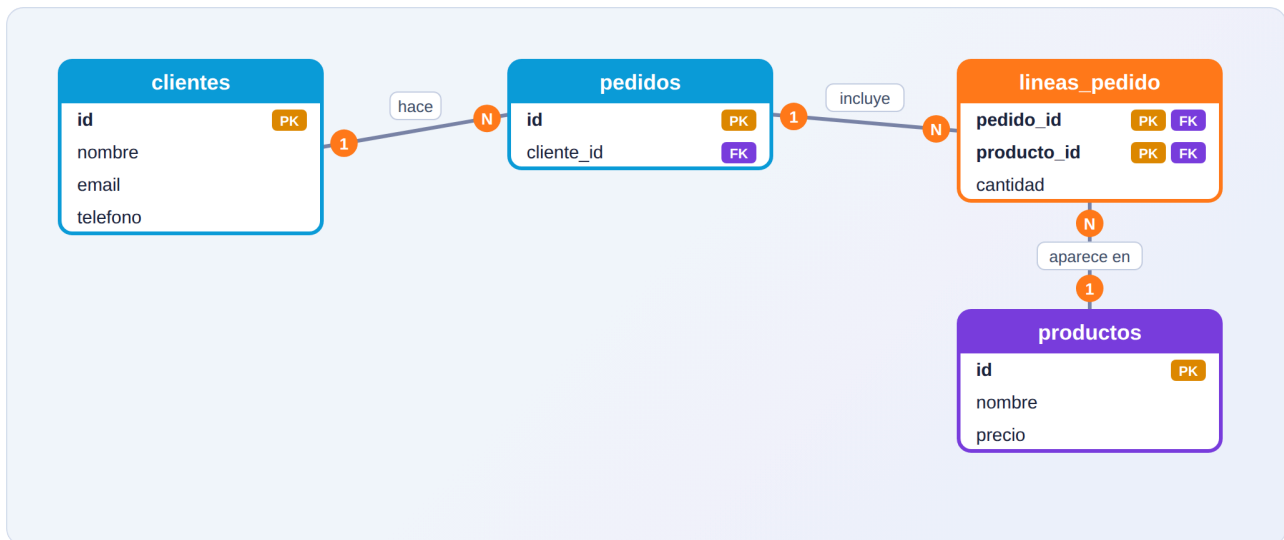


Figura 2.2-e. Diagrama de la estructura real, generado desde PostgreSQL.

Qué ves en la imagen. Las cuatro tablas con sus claves primarias (PK) y extranjeras (FK). La línea entre **productos** y **lineas_pedido** dibuja la relación «aparece en». Nada de esto se dibujó a mano: se leyó de la base de datos.

Ejemplo sencillo

«Cada cliente puede hacer varios pedidos»: *cliente* y *pedido* son entidades; *hacer* es la relación; «varios» indica que un cliente tiene muchos pedidos.

Ejemplo real

En una academia: «Los alumnos se matriculan en cursos. Cada curso lo imparte un profesor y tiene un máximo de 15 alumnos». Entidades: alumno, curso, profesor, matrícula. Regla: máximo de 15 alumnos por curso. Esa regla no se ve en ninguna tabla: deberá comprobarla la aplicación o la base de datos con una técnica más avanzada.

Profundizando

Reglas escondidas. Casi nunca aparecen escritas. «Cada habitación del hotel se reserva para unas fechas» esconde la regla «una habitación no puede estar reservada por dos personas en fechas que se solapan». Pregunta siempre: «¿qué no puede pasar nunca?».

Atributo frente a entidad: ciudad. En una agenda sencilla, «ciudad» es un atributo del contacto. En una empresa de reparto, que guarda zonas, tarifas y rutas por ciudad, es una entidad. Depende de lo que quieras guardar.

No todo se resuelve en la base de datos. Algunas reglas (como el aforo máximo) son más complejas y se resuelven con transacciones o restricciones avanzadas (Partes 5 y 6).

Ejercicio

1. Clasifica en entidades, atributos, relaciones y reglas: «Una biblioteca presta libros a socios. De cada libro se guarda el título y el autor. Un socio puede tener como máximo 3 libros prestados».

a la vez».

2. Escribe en SQL las tablas de ese análisis (sin la regla de los 3 libros, que es avanzada).
3. Encuentra una regla escondida en: «Cada habitación del hotel se reserva para unas fechas».

Solución

1. Entidades: libro, socio, préstamo. Atributos: título y autor del libro. Relaciones: la biblioteca *presta* libros a socios. Regla: como máximo 3 libros prestados a la vez por socio.
2. Por ejemplo:

```
CREATE TABLE socios (id integer PRIMARY KEY, nombre text NOT NULL);
CREATE TABLE libros (id integer PRIMARY KEY, titulo text NOT NULL, autor text NOT NULL);
CREATE TABLE prestamos (
  id integer PRIMARY KEY,
  socio_id integer NOT NULL REFERENCES socios (id),
  libro_id integer NOT NULL REFERENCES libros (id)
);
```

1. Una habitación no puede estar reservada por dos personas en fechas que se solapan.

Error habitual

Convertir en entidad algo que es un simple atributo (una tabla solo para el «color»), o guardar como atributo algo con vida propia (los pedidos dentro de una celda del cliente).

```
-- Mal: los pedidos como texto dentro del cliente
CREATE TABLE clientes_mal (id integer PRIMARY KEY, nombre text, pedidos text);
```

Buena práctica

Busca las **reglas escondidas** y escríbelas junto al diseño: son las que luego causan problemas si nadie las recordó.

Comprobación

Toma una descripción de tres o cuatro frases de un negocio que conozcas. Colorea con cuatro colores sus entidades, atributos, relaciones y reglas, y escribe las tablas. Si alguna palabra no cabe en ninguna categoría, probablemente no es relevante.

2.3 Datos obligatorios y datos opcionales

¿Qué es?

Un atributo es **obligatorio** cuando la fila no tiene sentido sin él, y **opcional** cuando puede faltar. Se decide para cada atributo, uno a uno. También se decide si debe ser **único** (no puede repetirse entre filas) y si tiene un **valor por defecto** (el que se usa si nadie indica otro).

¿Para qué sirve?

Protege la calidad de los datos. Si decides que el email es obligatorio, el sistema impedirá guardar clientes sin email. Si decides que es único, impedirá dos clientes con el mismo correo. Cada regla que dejas en manos de la base de datos es una regla que la aplicación no puede olvidar.

¿Por qué lo necesito?

Si todo es opcional, las tablas se llenan de huecos y las consultas dan resultados engañosos. Si todo es obligatorio, los usuarios no podrán guardar datos reales, porque a veces simplemente no los tienen. El equilibrio se decide caso por caso.

¿Cómo funciona?

Para cada atributo, cuatro preguntas:

1. **¿Puede existir la fila sin este dato?** Si no, es obligatorio.
2. **¿Se conoce siempre al crear la fila?** Si no, probablemente es opcional.
3. **¿Puede repetirse entre filas?** Si no, es único.
4. **¿Hay un valor razonable por defecto?** Por ejemplo, el estado de un pedido nuevo puede ser «pendiente».

Primero, sin código: decidir cada columna en una ficha

1. **Escribe la lista de atributos** de la entidad.
2. **Para cada uno, contesta las cuatro preguntas** y anótalo en una tabla.
3. **Prueba con un formulario:** ¿qué campos llevarían asterisco?

Alta de cliente

Nombre *

Luis Pérez

Email *

luis@ejemplo.com

Teléfono (opcional)

—

Guardar

* obligatorio

tienda · clientes

nombre obligatorio	email obligatorio · único	telefono opcional
Ana García	ana@ejemplo.com	600111222
Luis Pérez	luis@ejemplo.com	NULL
Marta Ruiz	marta@ejemplo.com	600333444

Figura 2.3-a. Un formulario y la tabla que lo guarda: los asteriscos marcan lo obligatorio.

Qué ves en la imagen. A la izquierda, un formulario de alta de cliente con nombre y email obligatorios (asterisco) y teléfono opcional. A la derecha, la tabla con las etiquetas «obligatorio», «único» y «opcional». La última fila (Luis) se guardó sin teléfono: su celda queda vacía (NULL).

Paso a paso: ahora con código

1. **Traduce cada decisión a una palabra clave:** obligatorio → `NOT NULL`; único → `UNIQUE`; por defecto → `DEFAULT valor`; opcional → no escribes nada.
2. **Crea la tabla** con esas palabras.
3. **Inserta una fila válida** dejando fuera las columnas opcionales: observa que se rellenan con NULL o con el valor por defecto.
4. **Intenta saltarte las reglas** (sin email; email repetido) y lee los errores.

Código y resultado

```
CREATE TABLE clientes_form (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,                -- obligatorio
  email   text NOT NULL UNIQUE,         -- obligatorio y único
  telefono text,                        -- opcional
  alta    date NOT NULL DEFAULT current_date -- obligatorio, con valor por defecto
);

INSERT INTO clientes_form (nombre, email) VALUES ('Luis Pérez',
'luis@ejemplo.com');

SELECT * FROM clientes_form;
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE clientes_form (
ensayo-# id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-# nombre text NOT NULL, -- obligatorio
ensayo-# email text NOT NULL UNIQUE, -- obligatorio y único
ensayo-# telefono text, -- opcional
ensayo-# alta date NOT NULL DEFAULT current_date -- obligatorio, con valor por defecto
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO clientes_form (nombre, email) VALUES ('Luis Pérez', 'luis@ejemplo.com');
INSERT 0 1

ensayo=# SELECT * FROM clientes_form;
 id | nombre | email | telefono | alta
-----+-----+-----+-----+-----
  1 | Luis Pérez | luis@ejemplo.com |  | 2026-10-03
(1 row)

```

Figura 2.3-b. Captura real: una tabla con columnas obligatorias, únicas, opcionales y con valor por defecto.

Qué ves en la imagen.

1. `nombre text NOT NULL`: obligatorio. `email text NOT NULL UNIQUE`: obligatorio y único. `telefono text`: opcional. `alta date NOT NULL DEFAULT current_date`: obligatorio, y si nadie la indica, vale la fecha de hoy.
2. El `INSERT` solo da nombre y email. El `SELECT` muestra que el `id` se ha rellenado solo (identidad automática, punto 3.4), el teléfono está vacío (NULL, opcional) y `alta` tiene la fecha de hoy gracias al valor por defecto.

```

INSERT INTO clientes_form (nombre) VALUES ('Sin Email');

INSERT INTO clientes_form (nombre, email) VALUES ('Otro Luis',
'luis@ejemplo.com');

```

```

psql — base de datos «ensayo»

ensayo=# INSERT INTO clientes_form (nombre) VALUES ('Sin Email');
ERROR: null value in column "email" of relation "clientes_form" violates not-null constraint
DETAIL: Failing row contains (2, Sin Email, null, null, 2026-10-03).

ensayo=# INSERT INTO clientes_form (nombre, email) VALUES ('Otro Luis', 'luis@ejemplo.com');
ERROR: duplicate key value violates unique constraint "clientes_form_email_key"
DETAIL: Key (email)=(luis@ejemplo.com) already exists.

```

Figura 2.3-c. Captura real: las reglas rechazan los datos que las incumplen.

Qué ves en la imagen. Sin email, el gestor responde `violates not-null constraint` y muestra la fila rechazada, donde `null` marca el email que falta. Con un email repetido, responde `violates unique constraint "clientes_form_email_key"` y el detalle `Key (email)=(luis@ejemplo.com) already exists`. En ambos casos **no se guarda nada**.

Ejemplo sencillo

En CLIENTES: `nombre` obligatorio; `email` obligatorio y único; `telefono` opcional; `alta` obligatoria, por defecto hoy.

Ejemplo real

En una tienda online, `fecha_envio` de un pedido es opcional (vacía hasta que sale del almacén), mientras que `estado` es obligatorio y por defecto «pendiente». Mirando la tabla se sabe que pedidos están sin enviar.

Profundizando

Empieza con pocos obligatorios. Es fácil endurecer una regla más tarde y difícil relajarla cuando ya hay datos incoherentes.

Los NULL tienen un coste. Cada columna opcional crea un caso especial en cada consulta (punto 1.6). Si una columna casi siempre está vacía, quizá sea otra entidad.

Único no es lo mismo que clave primaria. Una tabla puede tener muchas columnas únicas, pero una sola clave primaria.

Ejercicio

1. Para una tabla `productos` con `nombre`, `precio`, `descripcion` y `codigo_barras`, decide cuáles son obligatorias, únicas u opcionales.
2. Escribe el `CREATE TABLE` correspondiente.
3. ¿Qué problema aparece si se hace obligatorio el teléfono en el formulario de una tienda online?

Solución

1. `nombre`: obligatorio. `precio`: obligatorio. `descripcion`: opcional. `codigo_barras`: único y opcional (algunos productos no lo tienen, pero si existe no se repite).
2. Por ejemplo:

```
CREATE TABLE productos (
  id          integer PRIMARY KEY,
  nombre      text NOT NULL,
  precio      numeric(10, 2) NOT NULL,
  descripcion text,
  codigo_barras text UNIQUE
);
```

1. Algunos clientes no querrán darlo y abandonarán la compra, o inventarán un número, y la tabla se llenará de teléfonos falsos. Pide solo lo imprescindible.

Error habitual

Marcar un campo como obligatorio «porque sería ideal tenerlo» en lugar de «porque sin él la fila no tiene sentido». Los usuarios acabarán rellenándolo con basura.

```
-- Mal: obliga a dar un dato que no siempre existe  
telefono text NOT NULL  
-- Bien: opcional, y se valida solo si se da  
telefono text
```

Buena práctica

Obligatorio solo lo que la fila necesita para tener sentido; único solo lo que de verdad no puede repetirse; por defecto lo que casi siempre vale lo mismo.

Comprobación

Para cada columna de tu tabla escribe «obligatorio», «opcional», «único» o «con valor por defecto», con su justificación. Después intenta romper cada regla a propósito y comprueba que PostgreSQL la rechaza.

2.4 Caso guiado: «Quiero crear un CRM»

¿Qué es?

Un **CRM** (del inglés *Customer Relationship Management*, «gestión de la relación con los clientes») es una aplicación para llevar el seguimiento de las personas y empresas con las que trabajas: quiénes son, qué les has ofrecido, qué has hablado con ellas y qué tareas tienes pendientes.

Este punto aplica el método completo a un caso real, desde la frase hasta las tablas con datos.

¿Para qué sirve?

Es un ejercicio-modelo. Cuando termines, sabrás pasar de una frase tan vaga como «quiero un CRM» a una lista concreta de entidades y tablas, y podrás repetirlo con cualquier otro proyecto.

¿Por qué lo necesito?

Un CRM es uno de los proyectos más habituales que te pedirán, y mezcla casi todo lo que hay que saber: varias entidades relacionadas, historiales, estados, usuarios con permisos y tareas.

¿Cómo funciona?

Aplicamos los seis pasos del punto 2.1.

Paso 1: describir la idea. «Mi equipo de ventas trabaja con empresas y con las personas que trabajan en ellas. Queremos apuntar cada oportunidad de venta, saber en qué fase está, qué hemos hablado (llamadas, emails y notas) y qué tareas tenemos pendientes. Cada vendedor entra con su usuario y ve lo suyo».

Paso 2: entidades.

Entidad	Qué guarda
Usuario	Las personas del equipo que entran en el CRM
Empresa	Las organizaciones con las que se trabaja
Contacto	Las personas concretas, dentro de una empresa
Oportunidad	Una posible venta, con su importe y su fase
Estado (fase)	Cada paso del proceso: nuevo, en negociación, ganado, perdido
Tarea	Algo que hay que hacer, con fecha límite
Nota	Un apunte libre sobre una oportunidad
Llamada	El registro de una llamada con un contacto
Email	El registro de un correo enviado o recibido

Paso 3: atributos. Usuario (nombre, email, rol); empresa (nombre, sector, web, ciudad); contacto (nombre, email, teléfono, cargo); oportunidad (título, importe, fecha prevista de cierre); tarea (título, fecha límite, completada); nota (texto, fecha); llamada (fecha, duración, resumen); email (asunto, fecha, enviado o recibido).

Paso 4: relaciones. Una empresa tiene muchos contactos y muchas oportunidades. Cada oportunidad la lleva un usuario y está en un estado. Una oportunidad tiene muchas tareas, notas, llamadas y emails. Cada tarea se asigna a un usuario.

Paso 5: reglas. El email de un usuario es único. Una oportunidad siempre pertenece a una empresa. Una oportunidad tiene un único estado en cada momento. El importe no puede ser negativo. Un vendedor solo ve las oportunidades que tiene asignadas (se resolverá con permisos, Partes 16 y 17).

Paso 6: revisar con preguntas reales. ¿Qué oportunidades abiertas tiene cada vendedor? ¿Qué historial tiene una empresa? ¿Qué tareas vencen esta semana?



Figura 2.4-a. De la palabra «CRM» a las cosas que hay que guardar.

Qué ves en la imagen. En el centro la idea («CRM») y alrededor las entidades que salen de analizarla: principales en azul, de actividad en naranja y de organización en morado.

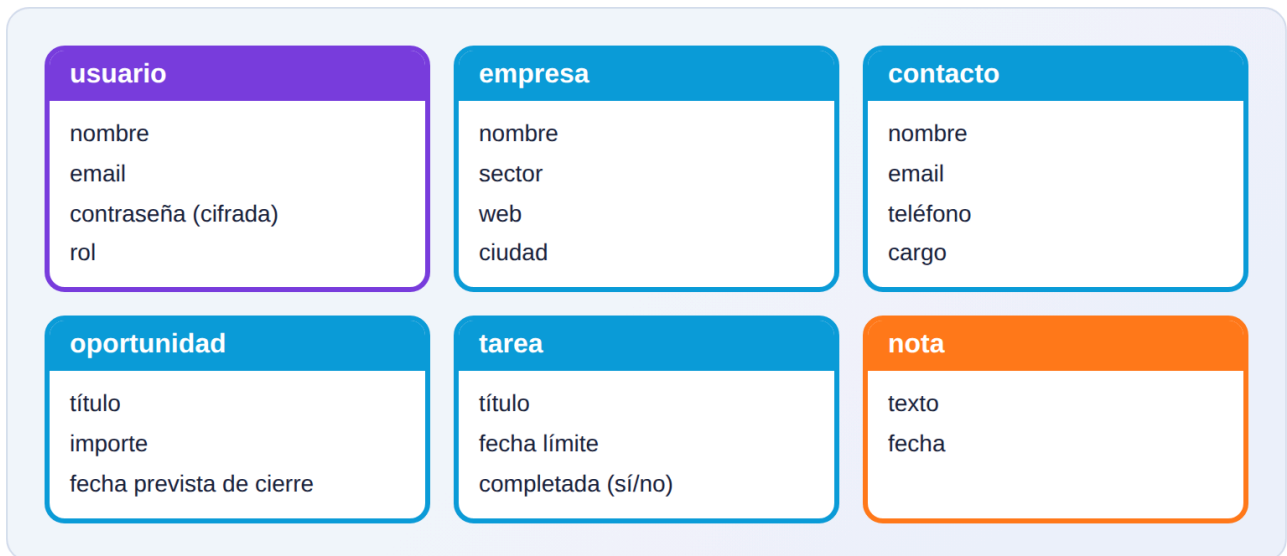


Figura 2.4-b. Atributos de las entidades principales del CRM.

Qué ves en la imagen. Una tarjeta por entidad con sus atributos. Cada tarjeta es el borrador de una futura tabla.

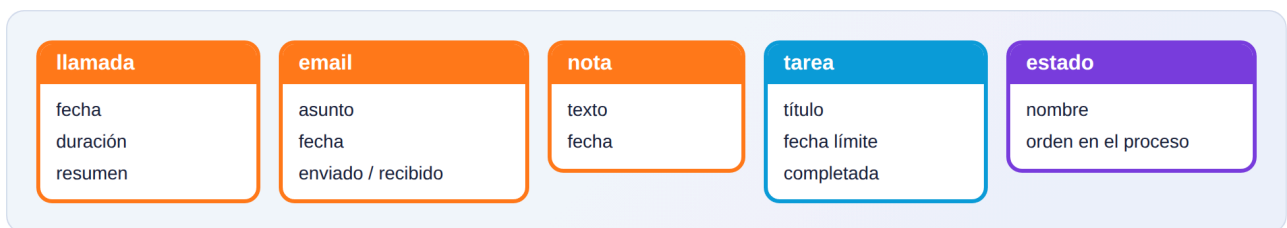


Figura 2.4-c. Atributos de las entidades de actividad y de los estados.

Qué ves en la imagen. Las entidades que registran lo que ocurre. Tienen pocos atributos propios; su valor está en las relaciones con las demás.

Primero, sin código: del análisis a las tablas con datos de ejemplo

Antes del SQL, construye las tablas **a mano**, con filas de ejemplo, igual que hiciste en la Parte 1:

1. **Una tabla por entidad**, con su **id**.
2. **Para cada relación uno a muchos**, pregunta «¿cuántos de un lado, cuántos del otro?» y pon la columna de enlace **en el lado «muchos»**.
3. **Rellena el enlace con el número** de la fila a la que apunta.
4. **Comprueba con preguntas del negocio** que puedes responderlas leyendo las tablas.

1 Una tabla por entidad, con su id

empresas			contactos	
id	nombre	sector	id	nombre
1	Acme S.L.	Industria	1	Lucía Mora
2	Norte Digital	Software	2	Pablo Ríos
3	Verde Sur	Agricultura	3	Irene Costa

Empresas y contactos son cosas distintas: cada una tiene su tabla. Las dos tienen id.

2 Pregunta: ¿cuántos contactos tiene una empresa? ¿Y cuántas empresas tiene un contacto?

Una empresa tiene **muchos** contactos; un contacto es de **una sola** empresa → uno a muchos.

Por tanto, la columna de enlace va en **contactos** (el lado «muchos»).

Regla: la columna de enlace siempre va en la tabla del lado «muchos».

3 Añade empresa_id a contactos y escribe el número de empresa

empresas			contactos		
id	nombre	sector	id	empresa_id	nombre
1	Acme S.L.	Industria	1	1	Lucía Mora
2	Norte Digital	Software	2	1	Pablo Ríos
3	Verde Sur	Agricultura	3	2	Irene Costa

Lucía y Pablo trabajan en Acme (1); Irene en Norte Digital (2). Verde Sur (3) aún no tiene contactos: es válido.

4 Repite el razonamiento con las oportunidades

empresas			oportunidades			
id	nombre		id	empresa_id	título	importe
1	Acme S.L.		1	1	Contrato anual	12000
2	Norte Digital		2	2	Rediseño web	4500
3	Verde Sur		3	1	Ampliación	3000

Una empresa tiene muchas oportunidades; cada oportunidad es de una empresa. La columna empresa_id va en oportunidades.

5 Comprueba con preguntas del negocio

¿Qué oportunidades tiene Acme? → las de empresa_id = 1 → «Contrato anual» y «Ampliación».

¿Qué contactos tiene Acme? → los de empresa_id = 1 → Lucía y Pablo.

Si una pregunta no se puede responder con las tablas, falta una tabla o una columna.

Figura 2.4-d. Sin código: las tablas del CRM construidas a mano, con datos de ejemplo.

Qué ves en la imagen. Paso 1: empresas y contactos como tablas separadas. Paso 2: el razonamiento que decide dónde va la columna de enlace. Paso 3: **empresa_id** añadida a **contactos** y rellenada con colores (Lucía y Pablo trabajan en Acme; Irene en Norte Digital; Verde Sur no tiene contactos y es válido). Paso 4: lo mismo con las oportunidades. Paso 5: dos preguntas del negocio respondidas leyendo las tablas. **Si sabes hacer esto en papel, el SQL es solo traducirlo.**

Paso a paso: ahora con código

1. **Conéctate a la base de datos** `crm` (créala antes con `CREATE DATABASE crm;` si no existe).
2. **Crea primero las tablas independientes:** `usuarios`, `estados`, `empresas`.
3. **Crea** `contactos`, que apunta a `empresas`.
4. **Crea** `oportunidades`, que apunta a tres tablas, y después `tareas`, `notas`, `llamadas` y `emails`, que apuntan a `oportunidades`.
5. **Comprueba con** `\dt` y con el diagrama.
6. **Inserta datos** (los mismos que usaste en papel) y haz preguntas del negocio con consultas.

Código y resultado

```
CREATE TABLE usuarios (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL,
  email   text NOT NULL UNIQUE,
  rol     text NOT NULL DEFAULT 'vendedor'
);

CREATE TABLE estados (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL UNIQUE,
  orden  integer NOT NULL
);

CREATE TABLE empresas (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL,
  sector text,
  web     text,
  ciudad text
);

CREATE TABLE contactos (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  empresa_id integer NOT NULL REFERENCES empresas (id),
  nombre       text NOT NULL,
  email        text,
  telefono     text,
  cargo        text
);
```

```

psql — base de datos «crm»

crm=# CREATE TABLE usuarios (
crm-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   nombre text NOT NULL,
crm-#   email   text NOT NULL UNIQUE,
crm-#   rol     text NOT NULL DEFAULT 'vendedor'
crm-# );
CREATE TABLE

crm=# CREATE TABLE estados (
crm-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   nombre  text NOT NULL UNIQUE,
crm-#   orden   integer NOT NULL
crm-# );
CREATE TABLE

crm=# CREATE TABLE empresas (
crm-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   nombre  text NOT NULL,
crm-#   sector  text,
crm-#   web     text,
crm-#   ciudad text
crm-# );
CREATE TABLE

crm=# CREATE TABLE contactos (
crm-#   id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   empresa_id integer NOT NULL REFERENCES empresas (id),
crm-#   nombre     text NOT NULL,
crm-#   email      text,
crm-#   telefono   text,
crm-#   cargo      text
crm-# );
CREATE TABLE

```

Figura 2.4-e. Captura real: las tablas independientes (`usuarios`, `estados`, `empresas`) y `contactos`.

Qué ves en la imagen. `usuarios`, `estados` y `empresas` no dependen de ninguna otra tabla, así que se crean primero. `contactos` lleva `empresa_id integer NOT NULL REFERENCES empresas (id)`: cada contacto pertenece obligatoriamente a una empresa. Las identidades automáticas (`GENERATED ALWAYS AS IDENTITY`) se explican en el punto 3.4. El `DEFAULT 'vendedor'` hace que un usuario nuevo sea vendedor salvo que se diga otra cosa.

```

CREATE TABLE oportunidades (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  empresa_id    integer NOT NULL REFERENCES empresas (id),
  usuario_id    integer NOT NULL REFERENCES usuarios (id),
  estado_id     integer NOT NULL REFERENCES estados (id),
  titulo        text NOT NULL,
  importe       numeric(12, 2) CHECK (importe >= 0),
  cierre_previsto date
);

CREATE TABLE tareas (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  oportunidad_id integer NOT NULL REFERENCES oportunidades (id),
  usuario_id    integer NOT NULL REFERENCES usuarios (id),
  titulo        text NOT NULL,
  vence         date,
  completada    boolean NOT NULL DEFAULT false
);

CREATE TABLE notas (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  oportunidad_id integer NOT NULL REFERENCES oportunidades (id),
  usuario_id    integer NOT NULL REFERENCES usuarios (id),
  texto         text NOT NULL,
  creada_en     timestampz NOT NULL DEFAULT now()
);

CREATE TABLE llamadas (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  oportunidad_id integer NOT NULL REFERENCES oportunidades (id),
  contacto_id   integer REFERENCES contactos (id),
  fecha         timestampz NOT NULL,
  minutos       integer CHECK (minutos >= 0),
  resumen       text
);

CREATE TABLE emails (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  oportunidad_id integer NOT NULL REFERENCES oportunidades (id),
  contacto_id   integer REFERENCES contactos (id),
  asunto        text NOT NULL,
  enviado       boolean NOT NULL, -- true = enviado; false = recibido
  fecha         timestampz NOT NULL
);

```

```

psql — base de datos «crm»

crm=# CREATE TABLE oportunidades (
crm-#   id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   empresa_id       integer NOT NULL REFERENCES empresas (id),
crm-#   usuario_id        integer NOT NULL REFERENCES usuarios (id),
crm-#   estado_id         integer NOT NULL REFERENCES estados (id),
crm-#   titulo            text NOT NULL,
crm-#   importe           numeric(12, 2) CHECK (importe >= 0),
crm-#   cierre_previsto   date
crm-# );
CREATE TABLE

crm=# CREATE TABLE tareas (
crm-#   id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   oportunidad_id    integer NOT NULL REFERENCES oportunidades (id),
crm-#   usuario_id        integer NOT NULL REFERENCES usuarios (id),
crm-#   titulo            text NOT NULL,
crm-#   vence            date,
crm-#   completada        boolean NOT NULL DEFAULT false
crm-# );
CREATE TABLE

crm=# CREATE TABLE notas (
crm-#   id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   oportunidad_id    integer NOT NULL REFERENCES oportunidades (id),
crm-#   usuario_id        integer NOT NULL REFERENCES usuarios (id),
crm-#   texto            text NOT NULL,
crm-#   creada_en         timestampz NOT NULL DEFAULT now()
crm-# );
CREATE TABLE

crm=# CREATE TABLE llamadas (
crm-#   id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   oportunidad_id    integer NOT NULL REFERENCES oportunidades (id),
crm-#   contacto_id       integer REFERENCES contactos (id),
crm-#   fecha            timestampz NOT NULL,
crm-#   minutos          integer CHECK (minutos >= 0),
crm-#   resumen          text
crm-# );
CREATE TABLE

crm=# CREATE TABLE emails (
crm-#   id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
crm-#   oportunidad_id    integer NOT NULL REFERENCES oportunidades (id),
crm-#   contacto_id       integer REFERENCES contactos (id),
crm-#   asunto            text NOT NULL,
crm-#   enviado           boolean NOT NULL, -- true = enviado; false = recibido
crm-#   fecha            timestampz NOT NULL
crm-# );
CREATE TABLE

```

Figura 2.4-f. Captura real: `oportunidades` y las tablas de actividad (`tareas`, `notas`, `llamadas`, `emails`).

Qué ves en la imagen. `oportunidades` tiene tres claves extranjeras (empresa, usuario y estado), todas obligatorias, y un `CHECK` que impide importes negativos. `tareas`, `notas`, `llamadas` y `emails` apuntan a `oportunidades`. En `llamadas` y `emails`, `contacto_id` es opcional (puede registrarse una llamada sin saber con quién fue). `enviado boolean` distingue correos enviados (`true`) de recibidos (`false`).

```
\dt
```

```
psql — base de datos «crm»

crm=# \dt

      List of relations
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | contactos      | table | postgres
 public | emails         | table | postgres
 public | empresas       | table | postgres
 public | estados        | table | postgres
 public | llamadas       | table | postgres
 public | notas          | table | postgres
 public | oportunidades  | table | postgres
 public | tareas         | table | postgres
 public | usuarios       | table | postgres
(9 rows)
```

Figura 2.4-g. Captura real: las nueve tablas creadas.

Qué ves en la imagen. `\dt` lista las nueve tablas del CRM.

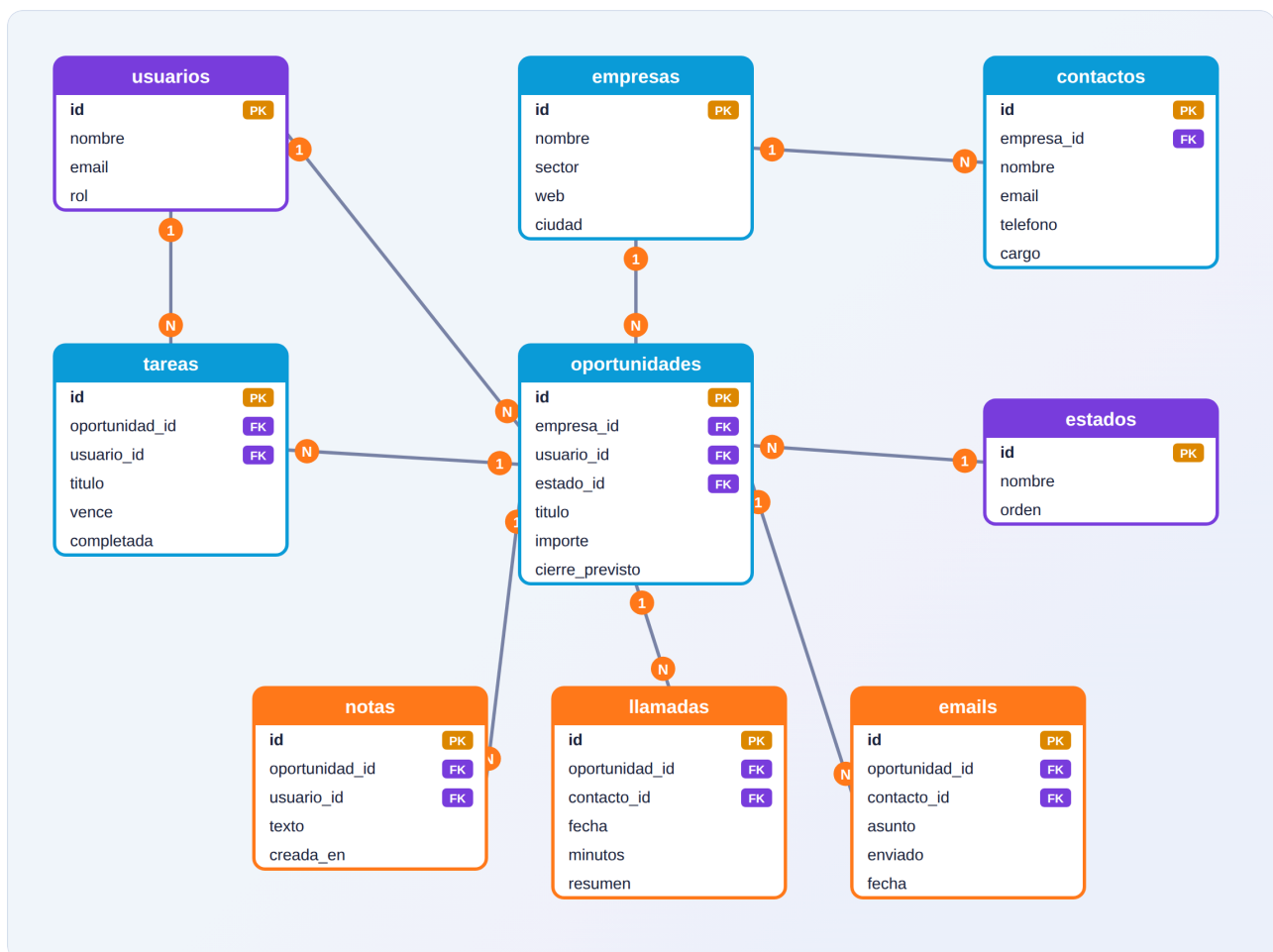


Figura 2.4-h. Diagrama del CRM generado desde la base de datos real.

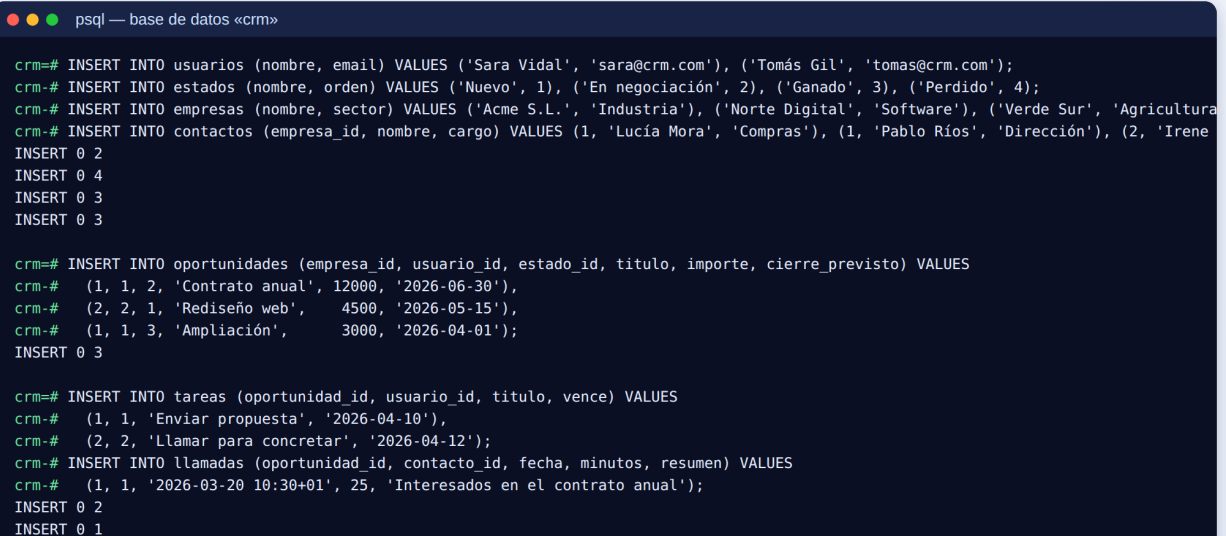
Qué ves en la imagen. Las nueve tablas con sus claves. Para que no se crucen las líneas, no se han dibujado las relaciones de **contactos** con **llamadas** y **emails** ni de **usuarios** con

`notas` ; sus claves extranjeras (FK) sí aparecen en cada tabla. `oportunidades` es el centro: de ella cuelgan las tareas, notas, llamadas y emails.

```
INSERT INTO usuarios (nombre, email) VALUES ('Sara Vidal', 'sara@crm.com'),
('Tomás Gil', 'tomas@crm.com');
INSERT INTO estados (nombre, orden) VALUES ('Nuevo', 1), ('En negociación', 2),
('Ganado', 3), ('Perdido', 4);
INSERT INTO empresas (nombre, sector) VALUES ('Acme S.L.', 'Industria'), ('Norte Digital', 'Software'), ('Verde Sur', 'Agricultura');
INSERT INTO contactos (empresa_id, nombre, cargo) VALUES (1, 'Lucía Mora', 'Compras'), (1, 'Pablo Ríos', 'Dirección'), (2, 'Irene Costa', 'Marketing');

INSERT INTO oportunidades (empresa_id, usuario_id, estado_id, titulo, importe, cierre_previsto) VALUES
(1, 1, 2, 'Contrato anual', 12000, '2026-06-30'),
(2, 2, 1, 'Rediseño web', 4500, '2026-05-15'),
(1, 1, 3, 'Ampliación', 3000, '2026-04-01');

INSERT INTO tareas (oportunidad_id, usuario_id, titulo, vence) VALUES
(1, 1, 'Enviar propuesta', '2026-04-10'),
(2, 2, 'Llamar para concretar', '2026-04-12');
INSERT INTO llamadas (oportunidad_id, contacto_id, fecha, minutos, resumen)
VALUES
(1, 1, '2026-03-20 10:30+01', 25, 'Interesados en el contrato anual');
```



```
psql — base de datos «crm»

crm=# INSERT INTO usuarios (nombre, email) VALUES ('Sara Vidal', 'sara@crm.com'), ('Tomás Gil', 'tomas@crm.com');
crm=# INSERT INTO estados (nombre, orden) VALUES ('Nuevo', 1), ('En negociación', 2), ('Ganado', 3), ('Perdido', 4);
crm=# INSERT INTO empresas (nombre, sector) VALUES ('Acme S.L.', 'Industria'), ('Norte Digital', 'Software'), ('Verde Sur', 'Agricultura');
crm=# INSERT INTO contactos (empresa_id, nombre, cargo) VALUES (1, 'Lucía Mora', 'Compras'), (1, 'Pablo Ríos', 'Dirección'), (2, 'Irene Costa', 'Marketing');
INSERT 0 2
INSERT 0 4
INSERT 0 3
INSERT 0 3

crm=# INSERT INTO oportunidades (empresa_id, usuario_id, estado_id, titulo, importe, cierre_previsto) VALUES
(1, 1, 2, 'Contrato anual', 12000, '2026-06-30'),
(2, 2, 1, 'Rediseño web', 4500, '2026-05-15'),
(1, 1, 3, 'Ampliación', 3000, '2026-04-01');
INSERT 0 3

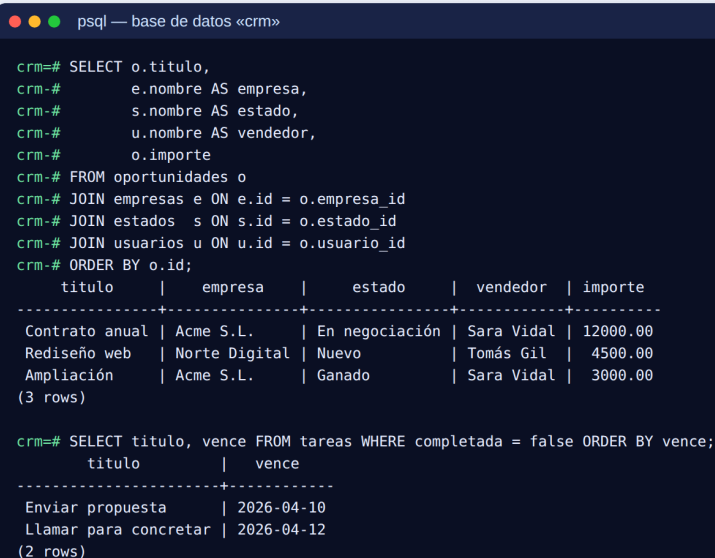
crm=# INSERT INTO tareas (oportunidad_id, usuario_id, titulo, vence) VALUES
(1, 1, 'Enviar propuesta', '2026-04-10'),
(2, 2, 'Llamar para concretar', '2026-04-12');
crm=# INSERT INTO llamadas (oportunidad_id, contacto_id, fecha, minutos, resumen) VALUES
(1, 1, '2026-03-20 10:30+01', 25, 'Interesados en el contrato anual');
INSERT 0 2
INSERT 0 1
```

Figura 2.4-i. Captura real: datos de ejemplo del CRM.

Qué ves en la imagen. Los `INSERT` guardan dos usuarios, cuatro estados, tres empresas, tres contactos, tres oportunidades, dos tareas y una llamada. Los números de enlace (`empresa_id`, `usuario_id`, `estado_id`) son los mismos que se rellenaron en papel. Las respuestas `INSERT 0 N` indican cuántas filas guardó cada sentencia.

```
SELECT o.titulo,
       e.nombre AS empresa,
       s.nombre AS estado,
       u.nombre AS vendedor,
       o.importe
FROM oportunidades o
JOIN empresas e ON e.id = o.empresa_id
JOIN estados s ON s.id = o.estado_id
JOIN usuarios u ON u.id = o.usuario_id
ORDER BY o.id;

SELECT titulo, vence FROM tareas WHERE completada = false ORDER BY vence;
```



```
psql — base de datos «crm»

crm=# SELECT o.titulo,
crm=#       e.nombre AS empresa,
crm=#       s.nombre AS estado,
crm=#       u.nombre AS vendedor,
crm=#       o.importe
crm=# FROM oportunidades o
crm=# JOIN empresas e ON e.id = o.empresa_id
crm=# JOIN estados s ON s.id = o.estado_id
crm=# JOIN usuarios u ON u.id = o.usuario_id
crm=# ORDER BY o.id;
 titulo | empresa | estado | vendedor | importe
-----+-----+-----+-----+-----
Contrato anual | Acme S.L. | En negociación | Sara Vidal | 12000.00
Rediseño web | Norte Digital | Nuevo | Tomás Gil | 4500.00
Ampliación | Acme S.L. | Ganado | Sara Vidal | 3000.00
(3 rows)

crm=# SELECT titulo, vence FROM tareas WHERE completada = false ORDER BY vence;
 titulo | vence
-----+-----
Enviar propuesta | 2026-04-10
Llamar para concretar | 2026-04-12
(2 rows)
```

Figura 2.4-j. Captura real: preguntas del negocio respondidas con consultas.

Qué ves en la imagen. La primera consulta responde «¿qué oportunidades hay, de qué empresa, en qué estado y de qué vendedor?», uniendo cuatro tablas con `JOIN`. La segunda responde «¿qué tareas pendientes hay y cuándo vencen?». Si el diseño no hubiera previsto la relación oportunidad-empresa-estado-vendedor, estas preguntas no se podrían contestar.

Ejemplo sencillo

Si el CRM fuera solo una libreta de empresas y contactos, bastarían dos entidades. Todo lo demás se añade por capas cuando el negocio lo pide.

Ejemplo real

Una consultora empieza con empresas, contactos y oportunidades. A los tres meses pide registrar llamadas y emails, y un mes después, tareas con recordatorios. Como las entidades están bien separadas, cada ampliación es una tabla nueva enlazada a las existentes, sin rehacer lo anterior.

Profundizando

Empieza por el núcleo. Querer diseñar el CRM «completo» el primer día acaba con cuarenta tablas que nadie usa. Empresas, contactos y oportunidades primero; el resto, cuando haga falta.

Multiempresa. Este CRM es de un solo equipo. Si quisieras que varias empresas lo usaran sin ver los datos de las demás (un SaaS), cada tabla necesitaría una columna que indique a qué empresa cliente pertenece. Se estudia en las Partes 16 y 20.

Contraseñas. La tabla `usuarios` no tiene contraseña a propósito: nunca se guarda una contraseña en claro, y cómo guardarla de forma segura se estudia en la Parte 16.

Ejercicio

1. ¿Qué entidad añadirías si el equipo quisiera registrar los productos que vende en cada oportunidad? ¿Qué tipo de relación tendría?
2. ¿Cuántas tareas puede tener una oportunidad? ¿Y cuántos usuarios puede tener una tarea asignada?
3. Escribe una consulta que devuelva el título y el importe de las oportunidades de la empresa Acme S.L. (usa `JOIN` con `empresas`).

Solución

1. Una entidad *producto* y una tabla intermedia `oportunidad_productos`, porque una oportunidad puede incluir varios productos y un producto aparecer en varias oportunidades (muchos a muchos, punto 3.3).
2. Una oportunidad puede tener muchas tareas. Cada tarea se asigna a un único usuario.
3. Por ejemplo:

```
SELECT o.titulo, o.importe
FROM oportunidades o
JOIN empresas e ON e.id = o.empresa_id
WHERE e.nombre = 'Acme S.L.';
```

Error habitual

Diseñar el CRM «completo» desde el primer día, o guardar el historial de llamadas y emails como texto dentro de la oportunidad.

```
-- Mal: el historial como texto; no se puede contar ni filtrar
ALTER TABLE oportunidades ADD COLUMN historial text;
```

Buena práctica

Verifica el diseño con **preguntas del negocio**. Si no puedes responderlas con las tablas que tienes, falta algo; si sobran tablas que no ayudan a responder ninguna, quítalas.

Comprobación

Sin mirar el texto, nombra seis entidades del CRM, una relación entre dos de ellas y dónde va su columna de enlace. Después comprueba con `\d oportunidades` que sus claves extranjeras coinciden con el diseño.

2.5 Otros casos de entrenamiento

¿Qué es?

Son tres análisis con el mismo método para que practiques: una **agenda de contactos**, una **tienda online** y un **sistema de reservas**. Cada uno incorpora un reto de diseño distinto.

¿Para qué sirve?

El análisis se aprende repitiéndolo. Cada caso te enseña un patrón que aparecerá en muchos proyectos: varios datos del mismo tipo por entidad (teléfonos), puentes entre entidades (líneas de pedido) y recursos con reglas de tiempo (reservas).

¿Por qué lo necesito?

Casi cualquier sistema que te pidan se parece a uno de estos tres patrones: guardar personas y sus datos, vender cosas, o reservar recursos en el tiempo.

¿Cómo funciona?

Para cada caso verás la descripción, las entidades, las relaciones y el reto. Intenta resolverlo tú antes de leer la solución.

Caso A: agenda de contactos. «Guardo contactos con sus teléfonos; cada contacto puede tener varios (móvil, casa, trabajo)». Entidades: contacto y teléfono. Reto: si el teléfono es una columna del contacto, ¿cuántos caben? `telefono1`, `telefono2` ... limita y desperdicia. La solución es una entidad aparte.

Caso B: tienda online. «Los clientes hacen pedidos con varios productos; queremos el precio al que se vendió cada uno». Entidades: cliente, producto, pedido, línea de pedido. Reto: el precio de un producto cambia, pero el pedido antiguo debe conservar el de aquel día; por eso la línea guarda su propio precio. Ya lo construiste en la Parte 3.

Caso C: reservas. «Un club alquila pistas por franjas de una hora; los socios reservan una pista para una fecha y hora». Entidades: socio, pista, reserva. Reto (regla): una pista no puede tener dos reservas a la misma hora.

Primero, sin código: resolver cada caso en papel

1. **Escribe las entidades** del caso.
2. **Pregunta en los dos sentidos** por cada relación y decide dónde va la columna de enlace.
3. **Escribe las reglas** que se desprenden del reto.
4. **Dibuja filas de ejemplo** y comprueba que caben.

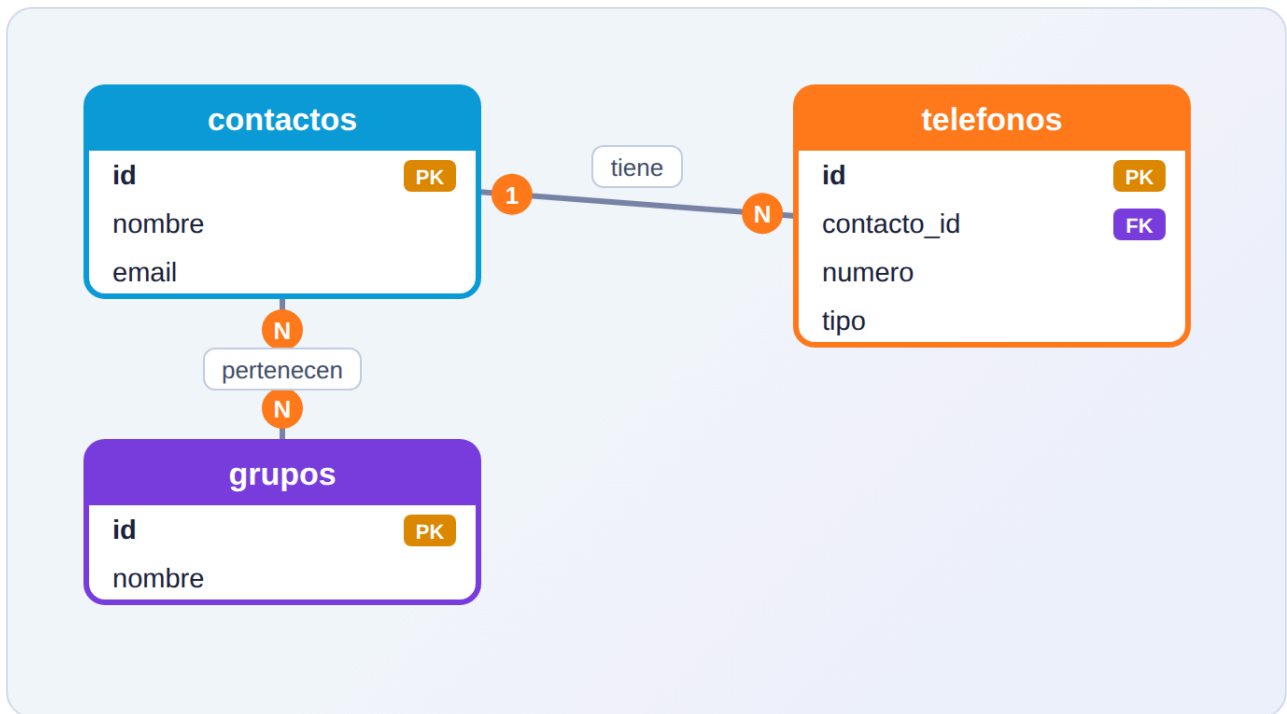


Figura 2.5-a. Caso A: agenda de contactos.

Qué ves en la imagen. Un contacto tiene muchos teléfonos (1 a N). Contactos y grupos se relacionan en ambos sentidos (N a N), que se resolverá con una tabla intermedia (punto 3.3).

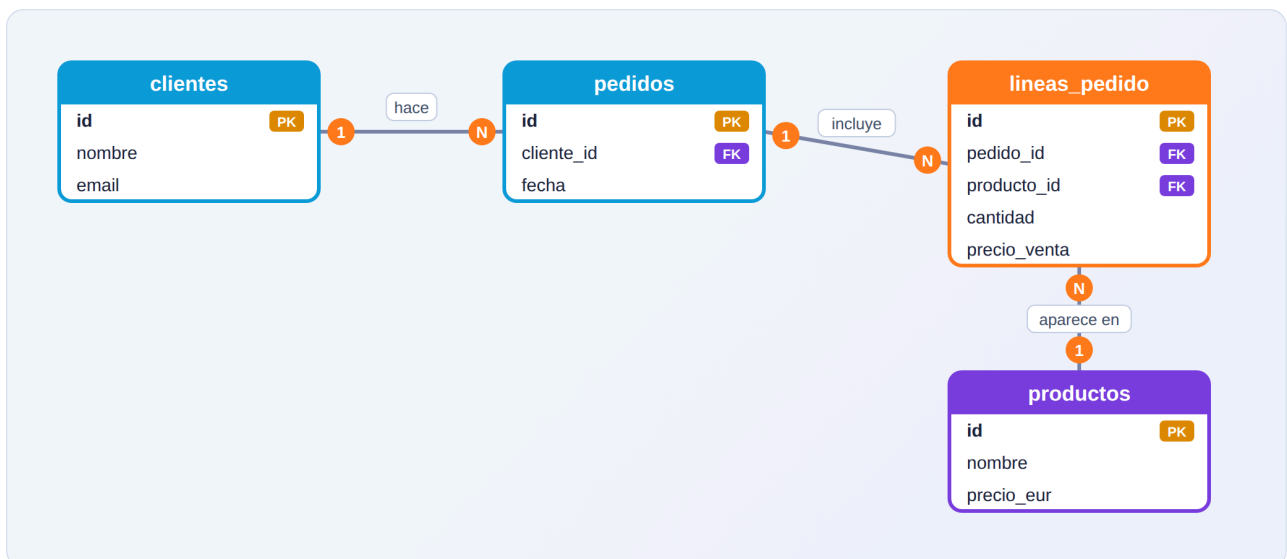


Figura 2.5-b. Caso B: tienda online.

Qué ves en la imagen. La línea de pedido es el puente entre pedido y producto: guarda cantidad y precio de venta.



Figura 2.5-c. Caso C: sistema de reservas.

Qué ves en la imagen. La reserva enlaza un socio con una pista en una fecha y hora. La regla del solape no se ve en el dibujo; hay que anotarla junto al diseño.

Paso a paso: ahora con código

1. **Caso A:** crea `contactos` y `telefonos` (con la clave extranjera en `telefonos`, el lado «muchos»), inserta un contacto con dos teléfonos y léelos.
2. **Caso C:** crea `socios`, `pistas` y `reservas`, y convierte la regla del reto en una restricción `UNIQUE` sobre pista, fecha y hora.
3. **Prueba la regla:** intenta reservar dos veces la misma pista a la misma hora.

Código y resultado

```

CREATE TABLE contactos (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL
);

CREATE TABLE telefonos (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  contacto_id integer NOT NULL REFERENCES contactos (id),
  numero       text NOT NULL,
  tipo         text NOT NULL CHECK (tipo IN ('móvil', 'casa', 'trabajo'))
);

INSERT INTO contactos (nombre) VALUES ('Ana García');
INSERT INTO telefonos (contacto_id, numero, tipo) VALUES (1, '600111222',
'móvil'), (1, '910000000', 'trabajo');

SELECT c.nombre, t.tipo, t.numero
FROM contactos c
JOIN telefonos t ON t.contacto_id = c.id;
  
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE contactos (
ensayo-#   id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   nombre text NOT NULL
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE telefonos (
ensayo-#   id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   contacto_id integer NOT NULL REFERENCES contactos (id),
ensayo-#   numero     text NOT NULL,
ensayo-#   tipo       text NOT NULL CHECK (tipo IN ('móvil', 'casa', 'trabajo'))
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO contactos (nombre) VALUES ('Ana García');
ensayo-# INSERT INTO telefonos (contacto_id, numero, tipo) VALUES (1, '600111222', 'móvil'), (1, '910000000', 'trabajo');
INSERT 0 1
INSERT 0 2

ensayo=# SELECT c.nombre, t.tipo, t.numero
ensayo-# FROM contactos c
ensayo-# JOIN telefonos t ON t.contacto_id = c.id;
 nombre | tipo | numero
-----+-----+-----
 Ana García | móvil | 600111222
 Ana García | trabajo | 910000000
(2 rows)

```

Figura 2.5-d. Captura real: el caso A en SQL. Un contacto con dos teléfonos en filas distintas.

Qué ves en la imagen. En lugar de dos columnas de teléfono, hay una tabla `telefonos` con una fila por número, enlazada con `contacto_id`. `CHECK (tipo IN ('móvil', 'casa', 'trabajo'))` limita los tipos permitidos. La consulta muestra a Ana García con sus dos teléfonos, cada uno en una fila. Si mañana Ana tiene un tercero, es una fila más, sin tocar la estructura.

```
CREATE TABLE socios (  
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre text NOT NULL  
);  
  
CREATE TABLE pistas (  
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre text NOT NULL  
);  
  
CREATE TABLE reservas (  
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  socio_id integer NOT NULL REFERENCES socios (id),  
  pista_id integer NOT NULL REFERENCES pistas (id),  
  fecha      date NOT NULL,  
  hora       time NOT NULL,  
  UNIQUE (pista_id, fecha, hora)  -- regla: una pista no se reserva dos veces a  
la misma hora  
);  
  
INSERT INTO socios (nombre) VALUES ('Marta'), ('Luis');  
INSERT INTO pistas (nombre) VALUES ('Pista 1');  
INSERT INTO reservas (socio_id, pista_id, fecha, hora) VALUES (1, 1, '2026-04-  
05', '18:00');  
  
INSERT INTO reservas (socio_id, pista_id, fecha, hora) VALUES (2, 1, '2026-04-  
05', '18:00');
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE socios (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   nombre text NOT NULL
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE pistas (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   nombre text NOT NULL
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE reservas (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   socio_id integer NOT NULL REFERENCES socios (id),
ensayo-#   pista_id integer NOT NULL REFERENCES pistas (id),
ensayo-#   fecha   date NOT NULL,
ensayo-#   hora    time NOT NULL,
ensayo-#   UNIQUE (pista_id, fecha, hora) -- regla: una pista no se reserva dos veces a la misma hora
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO socios (nombre) VALUES ('Marta'), ('Luis');
ensayo=# INSERT INTO pistas (nombre) VALUES ('Pista 1');
ensayo=# INSERT INTO reservas (socio_id, pista_id, fecha, hora) VALUES (1, 1, '2026-04-05', '18:00');
INSERT 0 2
INSERT 0 1
INSERT 0 1

ensayo=# INSERT INTO reservas (socio_id, pista_id, fecha, hora) VALUES (2, 1, '2026-04-05', '18:00');
ERROR:  duplicate key value violates unique constraint "reservas_pista_id_fecha_hora_key"
DETAIL:  Key (pista_id, fecha, hora)=(1, 2026-04-05, 18:00:00) already exists.

```

Figura 2.5-e. Captura real: el caso C en SQL. La regla del reto convertida en restricción.

Qué ves en la imagen. `UNIQUE (pista_id, fecha, hora)` es una restricción sobre la **combinación** de tres columnas: no puede haber dos filas con la misma pista, fecha y hora. La primera reserva (Marta, pista 1, 5 de abril a las 18:00) se guarda; la segunda, de Luis, para la misma pista, fecha y hora, es rechazada con `duplicate key value violates unique constraint "reservas_pista_id_fecha_hora_key"` y el detalle de la combinación repetida. El caso B (tienda) ya lo construiste y ejecutaste en el punto 3.3.

Ejemplo sencillo

En la agenda, Ana García tiene dos teléfonos: dos filas en la tabla de teléfonos, ambas enlazadas con Ana.

Ejemplo real

Una tienda de ropa guarda en cada línea de pedido el precio de venta y no solo la referencia al producto. Así, tres años después, puede reproducir la factura de un pedido aunque el producto haya subido de precio.

Profundizando

Solapes de tiempo reales. `UNIQUE (pista, fecha, hora)` funciona para franjas fijas de una hora. Si las reservas pueden durar cualquier tiempo, la regla «no se solapan» se resuelve con un tipo de restricción más avanzado de PostgreSQL (restricciones de exclusión con rangos de tiempo, Parte 6).

Patrones. Reconoce los patrones: «varios datos del mismo tipo por entidad» (teléfonos), «puente entre dos entidades» (líneas de pedido), «recurso con reglas de tiempo» (reservas). Una vez identificados, reutilizas la solución.

Muchos a muchos con grupos. Contactos y grupos (caso A) necesitan una tabla intermedia, como `lineas_pedido`. Se resuelve en el punto 3.3.

Ejercicio

1. Añade al caso C una entidad para que cada pista tenga un precio por hora distinto según el día de la semana. ¿Qué atributos tendría?
2. En el caso B, ¿por qué no basta con guardar el producto y la cantidad en el pedido?
3. Analiza este caso: «Un taller mecánico guarda los coches de sus clientes y las reparaciones que les hace». Escribe entidades, relaciones, una regla y el SQL de las tablas.

Solución

1. Una entidad *tarifa* con atributos como pista, día de la semana y precio por hora. Una pista tiene varias tarifas.
2. Porque se pierde el precio al que se vendió: si el producto cambia de precio, el pedido antiguo mostraría un precio que nunca se pagó.
3. Entidades: cliente, coche, reparación. Relaciones: un cliente tiene varios coches; un coche tiene varias reparaciones. Regla: la matrícula es única.

```
CREATE TABLE clientes (id integer PRIMARY KEY, nombre text NOT NULL);
CREATE TABLE coches (
  id integer PRIMARY KEY,
  cliente_id integer NOT NULL REFERENCES clientes (id),
  matricula text NOT NULL UNIQUE
);
CREATE TABLE reparaciones (
  id integer PRIMARY KEY,
  coche_id integer NOT NULL REFERENCES coches (id),
  fecha date NOT NULL,
  detalle text
);
```

Error habitual

Copiar el diseño de un caso a otro sin comprobar que encaja. Una tienda y un sistema de reservas parecen similares, pero tienen reglas distintas.

```
-- Mal: columnas repetidas en lugar de una tabla de teléfonos
CREATE TABLE contactos_mal (id integer PRIMARY KEY, nombre text, telefono1 text,
telefono2 text, telefono3 text);
```

Buena práctica

Para cada caso nuevo, empieza por buscar qué patrón se repite y qué regla específica lo distingue.

Comprobación

Elige un negocio cercano (una peluquería, un colegio, una biblioteca). En diez minutos escribe sus entidades, relaciones y una regla importante, y escribe el SQL de sus tablas.

2.6 Plantilla reutilizable de análisis de requisitos de datos

¿Qué es?

Una **plantilla de requisitos de datos** es un documento sencillo, igual para todos los proyectos, donde recoges todo lo que has decidido en el análisis. Es el puente entre la idea y el diseño de tablas. Se rellena una ficha por entidad y una lista de relaciones y reglas.

¿Para qué sirve?

Para dejar el análisis por escrito de forma ordenada: para revisar con el cliente, para no olvidar decisiones y para que otra persona (o tú dentro de seis meses) entienda el proyecto sin preguntar.

¿Por qué lo necesito?

Los análisis hechos «de cabeza» se olvidan o se deforman. Una plantilla obliga a contestar siempre las mismas preguntas, y las preguntas olvidadas suelen ser las que luego causan problemas.

¿Cómo funciona?

La plantilla tiene cuatro partes:

1. **Descripción del proyecto:** tres a cinco frases.
2. **Ficha de cada entidad:** atributos, y para cada uno tipo, obligatorio, único y notas.
3. **Lista de relaciones** con su cardinalidad (uno a uno, uno a muchos, muchos a muchos).
4. **Lista de reglas y preguntas de negocio** que el diseño debe poder responder.

Primero, sin código: rellenar la plantilla

Ficha de la entidad CLIENTE de la tienda:

Atributo	Tipo	Obligatorio	Único	Notas
id	número entero	sí	sí	Identificador automático
nombre	texto	sí	no	Nombre completo
email	texto	sí	sí	Se usa para iniciar sesión
telefono	texto	no	no	Con prefijo internacional si se conoce
alta	fecha	sí	no	Por defecto, la fecha de hoy

Ficha de entidad: CLIENTE

Descripción: persona que compra en la tienda.

Atributo	Tipo	Obligatorio	Único	Notas
id	entero	sí	sí	Identificador automático
nombre	texto	sí	no	Nombre completo
email	texto	sí	sí	Para iniciar sesión
telefono	texto	no	no	Con prefijo si se conoce
alta	fecha	sí	no	Por defecto, hoy

Relaciones: un cliente hace muchos pedidos (1 a N).

Reglas: el email no se repite · el nombre no puede estar vacío.

Preguntas de negocio: ¿cuántos clientes nuevos hay por mes? ¿qué compró cada cliente?

Figura 2.6-a. Ficha de entidad rellena para CLIENTE, con relaciones, reglas y preguntas de negocio.

Qué ves en la imagen. La ficha completa tal y como se vería en un documento de análisis: nombre de la entidad, descripción, tabla de atributos y, abajo, relaciones, reglas y preguntas de negocio. Se rellena una por cada entidad.

Paso a paso: ahora con código

La plantilla tiene una ventaja: **se puede comprobar contra la base de datos real**. Una vez creada la tabla, PostgreSQL puede devolverte su «ficha» leyendo el catálogo:

1. **Crea la tabla** a partir de la plantilla.
2. **Consulta** `information_schema.columns`, que describe las columnas de cada tabla.
3. **Compara** la respuesta con tu ficha: deben coincidir.

Código y resultado

```
SELECT column_name AS columna,
       data_type   AS tipo,
       is_nullable AS "¿admite vacío?",
       column_default AS por_defecto
FROM information_schema.columns
WHERE table_name = 'usuarios'
ORDER BY ordinal_position;
```

```

psql — base de datos «crm»

crm=# SELECT column_name AS columna,
crm=#         data_type AS tipo,
crm=#         is_nullable AS "¿admite vacío?",
crm=#         column_default AS por_defecto
crm=# FROM information schema.columns
crm=# WHERE table_name = 'usuarios'
crm=# ORDER BY ordinal_position;
columna | tipo  | ¿admite vacío? | por_defecto
-----+-----+-----+-----
id       | integer | NO           |
nombre   | text   | NO           |
email    | text   | NO           |
rol      | text   | NO           | 'vendedor'::text
(4 rows)

```

Figura 2.6-b. Captura real: la «ficha» de la tabla `usuarios` del CRM, leída del catálogo de PostgreSQL.

Qué ves en la imagen. `information_schema.columns` es una tabla del sistema que describe las columnas de todas las tablas. La consulta pide, para `usuarios`, el nombre de cada columna, su tipo, si admite vacío (`NO` significa que es obligatoria) y su valor por defecto. Es exactamente la información de la ficha: `rol` es obligatoria y tiene por defecto `'vendedor'`. Si tu ficha dice otra cosa, el diseño y la base de datos no coinciden.

Ejemplo sencillo

Antes de crear la tabla `clientes`, rellenas la ficha. Al crearla, ejecutas la consulta anterior para `clientes` y comprobas que coincide.

Ejemplo real

Una agencia usa siempre la misma plantilla en las reuniones con clientes. Rellenarla en directo hace que el cliente descubra, sin tecnicismos, datos en los que no había pensado («¿y las devoluciones?«).

Profundizando

Documentación viva. Guarda la plantilla junto al código y actualízala con cada cambio. Si puedes, genera partes de ella desde la base de datos (como en la figura) para que nunca difieran.

Preguntas de negocio como pruebas. Cada pregunta de negocio de la plantilla es, en la práctica, una consulta de prueba: si no se puede escribir, el diseño falla.

Más allá de la plantilla. En proyectos grandes se añaden volúmenes esperados (cuántas filas), permisos y requisitos de rendimiento.

Ejercicio

1. Rellena una ficha para la entidad PRODUCTO de la tienda.
2. Escribe las relaciones y reglas de la tienda en forma de lista.
3. Crea la tabla `productos` según tu ficha y comprueba con `information_schema.columns` que coincide.

Solución

1. `id` (entero, obligatorio, único); `nombre` (texto, obligatorio); `precio_eur` (decimal, obligatorio, no negativo); `stock` (entero, obligatorio, por defecto 0); `activo` (verdadero/falso, por defecto verdadero).
2. Relaciones: cliente–pedido (1 a N); pedido–línea (1 a N); producto–línea (1 a N). Reglas: email único; precio no negativo; cantidad mayor que cero.
3. Con la consulta de la figura cambiando `'usuarios'` por `'productos'`.

Error habitual

Rellenar la plantilla una vez, al principio, y no actualizarla. Un análisis desactualizado engaña más que ayuda.

```
-- Comprobar si la ficha sigue coincidiendo con la tabla real
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'clientes';
```

Buena práctica

Guarda la plantilla junto al código y actualízala cada vez que cambie el diseño.

Comprobación

Con la plantilla de un proyecto, otra persona debería poder responder, sin preguntarte, qué datos se guardan, cuáles son obligatorios y qué reglas existen.

Resumen de la Parte 2

- Antes de crear tablas se piensa: describir, subrayar, anotar, relacionar, escribir reglas y revisar con preguntas reales.
- Las entidades son las cosas que se guardan (tablas); los atributos son sus características (columnas); las relaciones las conectan; las reglas limitan lo que está permitido.
- Cada atributo se decide como obligatorio u opcional, único o repetible, con o sin valor por defecto, y PostgreSQL hace cumplir esas decisiones.
- Las tablas y sus relaciones se construyen primero a mano, con datos de ejemplo, y después en SQL: la columna de enlace va en el lado «muchos».
- Un CRM se analiza por capas: núcleo (empresas, contactos, oportunidades) y después historial y tareas.
- Los patrones se repiten: datos múltiples por entidad (teléfonos), puentes (líneas de pedido) y recursos con tiempo (reservas).
- Una plantilla de requisitos deja el análisis por escrito y puede comprobarse contra la base de datos real.

Glosario de la Parte 2

Término	Significado
Entidad	Cosa de la que se guardan datos; acabará siendo una tabla
Atributo	Característica de una entidad; acabará siendo una columna
Relación	Conexión entre dos entidades
Regla	Condición que los datos deben cumplir
Obligatorio / opcional	Dato sin el cual la fila no tiene sentido / que puede faltar
Único	Dato que no puede repetirse entre filas
Valor por defecto	Valor que se usa si nadie indica otro
CRM	Aplicación para gestionar la relación con clientes
Oportunidad	Una posible venta, con importe y fase
Línea de pedido	Un producto con su cantidad y precio dentro de un pedido
Requisitos de datos	Descripción escrita de lo que se debe guardar y de sus reglas
Catálogo	Tablas internas donde PostgreSQL describe la base de datos

Mini examen de la Parte 2

1. Nombra los seis pasos del método para pasar de una idea a los datos.
2. ¿Qué diferencia hay entre una entidad y un atributo? Pon un ejemplo.
3. En «un cliente hace varios pedidos», ¿qué palabra indica la relación y en qué tabla va la columna de enlace?
4. ¿Cuándo es obligatorio un atributo y qué palabra clave de SQL lo impone?
5. ¿Por qué es mala idea que el teléfono del cliente sea siempre obligatorio en una tienda online?
6. Nombra cinco entidades de un CRM.
7. ¿Por qué la línea de pedido guarda su propio precio?
8. ¿Cómo se convierte en SQL la regla «una pista no se reserva dos veces a la misma hora»?
9. ¿Qué partes tiene la plantilla de requisitos?
10. ¿Cómo puedes comprobar que tu ficha coincide con la tabla real?

Respuestas

1. Describir la idea; subrayar entidades; anotar atributos; buscar relaciones; escribir reglas; revisar con preguntas reales.

2. Una entidad es una cosa de la que se guardan varios datos (cliente); un atributo es una característica de ella (el email del cliente).
3. La palabra «hace». La columna de enlace (`cliente_id`) va en `pedidos`, el lado «muchos».
4. Cuando la fila no tiene sentido sin él; se impone con `NOT NULL`.
5. Algunos clientes no querrán darlo y abandonarán la compra o inventarán un número falso.
6. Por ejemplo: usuario, empresa, contacto, oportunidad y tarea.
7. Porque el precio del producto puede cambiar, y el pedido antiguo debe conservar el precio al que se vendió.
8. Con una restricción `UNIQUE (pista_id, fecha, hora)`.
9. Descripción del proyecto, ficha de cada entidad, relaciones, y reglas con preguntas de negocio.
10. Consultando `information_schema.columns` para esa tabla y comparando la respuesta con la ficha.

Si has acertado 8 o más, estás listo para la Parte 3. Si no, repasa los puntos de las preguntas falladas.